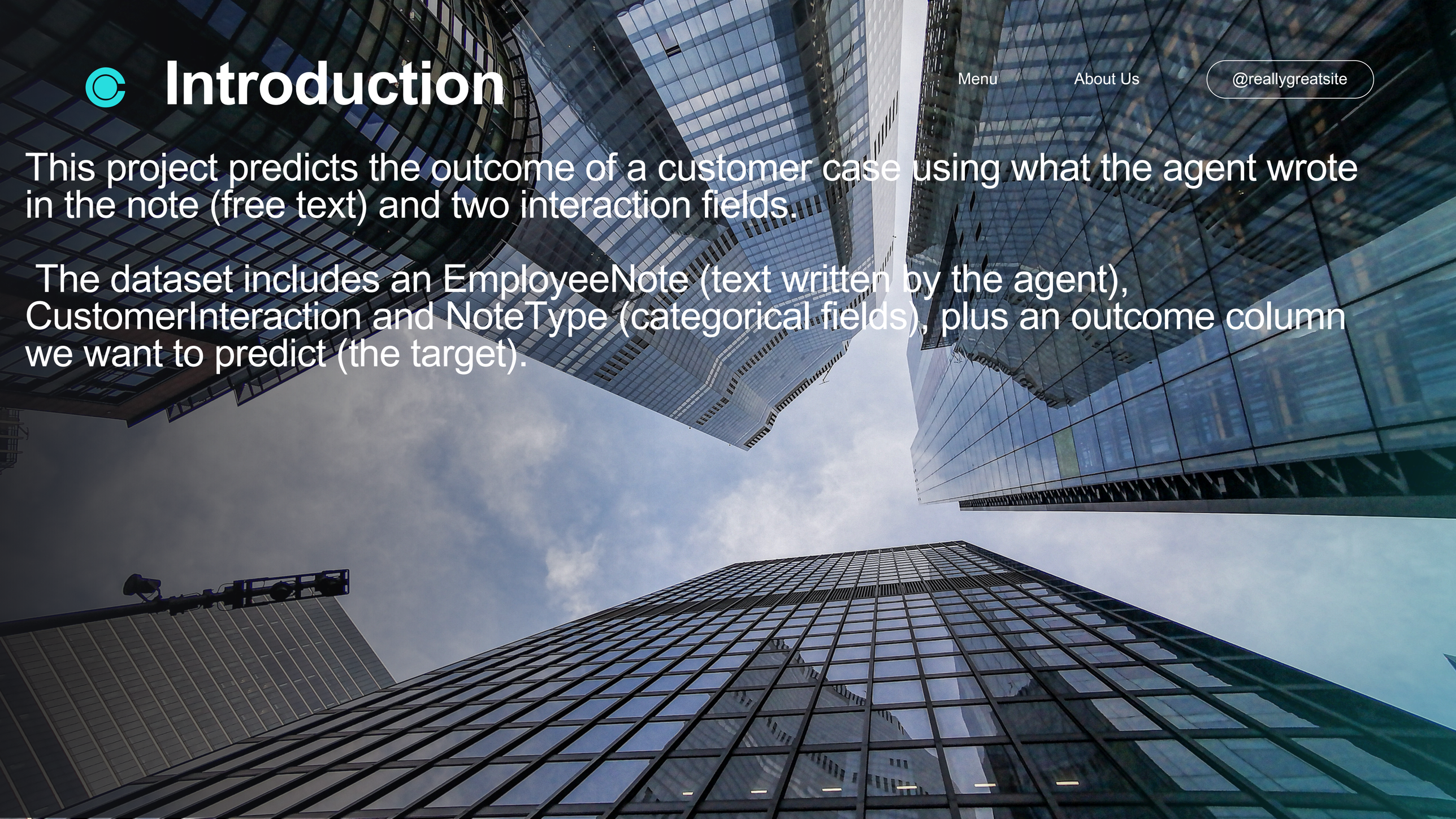
Classifying Customer Outcomes from Notes & Interactions

Introduction

This project predicts the outcome of a customer case using what the agent wrote in the note (free text) and two interaction fields.

The dataset includes an EmployeeNote (text written by the agent), CustomerInteraction and NoteType (categorical fields), plus an outcome column we want to predict (the target).



Business goal (plain)

use what agents write and select in the system to quickly flag cases that need attention (e.g., “Pending”) and speed up resolution. If we can detect risky cases early, teams can prioritize callbacks and reduce backlog.

ginyard

Data & Target (what we model)

The target (y) is the case outcome column (e.g., “Pending”, “Resolved”, …) taken from the dataset’s 6th column. We convert it to numeric labels internally so the model can learn.

key highlights

EmployeeNote: the agent’s written note (free text).

CustomerInteraction: type of interaction (categorical).

NoteType: type/category of the note (categorical).

Turning raw columns into model-ready signals

Text features (EmployeeNote):

We build a vocabulary of words and short word-pairs (uni-grams and bi-grams) from the notes and convert each note into a TF-IDF vector.

Categorical features (CustomerInteraction, NoteType):

We one-hot encode these fields so each category becomes a 0/1 column.

ColumnTransformer:

We apply both steps in one pipeline so training and future predictions receive the exact same preprocessing.

ginyard

Model: Logistic Regression on sparse text+category features

We train a Logistic Regression classifier on the transformed data.

Why LR works well here: it’s fast, stable on high-dimensional sparse text features (TF-IDF), and gives strong baselines. It learns weighted combinations of words/phrases and categories that push a case toward each class.

Result: Test accuracy is about 98.77%.

PHASE 2 after modeliing Process

Turning raw columns in to model-ready signals

Turning raw columns in to model-ready signals

We explored a simple, explainable signal: how long the note is.

We computed note_len = number of words in EmployeeNote.

We computed
We looked at the Pending rate across four length buckets (quartiles).

The observed Pending rates were approximately:

- 3–7 words: 27.9% pending
- 7–9 words: 27.7% pending
- 9–19 words: 10.2% pending
- 19–110 words: 21.8% pending

What this means (plain): very short notes tend to have higher pending risk (possibly rushed or incomplete), medium-length notes (about 9–19 words) are least risky, and very long notes rise again (complex cases). It’s a mild U-shaped pattern.

To make this usable, we defined a simple rule:

- RED (high risk): note_len ≥ 20
- YELLOW (moderate): note_len ≤ 9
- GREEN (low): everything in between

This rule is easy to explain to teams and can be shown in dashboards as an **early risk flag** even before the full model runs, or used alongside the model to prioritize reviews.

ginyard

How to use this in the business

- Use the Logistic Regression model to auto-label incoming cases by outcome with ~98.8% accuracy, and route likely “Pending” cases to a priority queue.

- Show the note-length risk color next to each case for a quick, human-friendly signal: short notes (YELLOW) may need follow-up; very long notes (RED) may indicate complex issues; mid-length (GREEN) are generally safer.

- Combine both: the model gives the main decision; the color is a simple explanation and a back-up heuristic for triage when notes are incomplete or systems are slow.

ginyard

Menu About Us @reallygreatsite

thank you

123 Anywhere St.

+123-456-7890

@reallygreatsite